

Subprograms, Parameters, and Closures

Liting Zhang

University of West Florida

Overview

- Subprograms: procedures vs functions
- First-class vs second-class functions
- Parameter passing models and common language behaviors
- Defaults, named arguments, and API design
- Closures: lexical environments and variable capture
- Higher-order functions: callbacks and lambdas
- Industry angle: callback hell vs async/await; closures in UI and React

Learning goals

After this lecture, you should be able to

- Distinguish procedures from functions and explain why the distinction matters
- Identify first-class function features and compare TypeScript, Java, Python support
- Predict effects of different parameter passing strategies
- Use default parameters and named arguments to design readable APIs
- Explain what a closure is and what it means to capture a variable
- Diagnose common closure bugs (loop capture, stale state) and apply fixes
- Refactor callback-based async code into async/await with correct error handling

Core idea: subprograms as reusable behavior

A subprogram is a named unit of executable code that can be invoked.

Common reasons to use subprograms

- Abstraction: hide details behind a simple interface
- Reuse: call the same logic from many places
- Testability: isolate logic to test it independently
- Composition: build larger behavior from smaller pieces

In practice, subprograms include functions, procedures, methods, constructors, lambdas, and callbacks.

Procedures and Functions

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

Procedures vs functions

Procedure

- Emphasizes performing an action
- May not return a meaningful value
- Often used for I/O, logging, mutation of state

Function

- Emphasizes computing and returning a value
- Ideally depends only on inputs (pure), but real code may still have side effects

In many modern languages, the syntax does not enforce a strict separation, so the distinction is about design intent.

```
1 function loginUser(userId: string): void {  
2     console.log("login", userId); // procedure-like  
3 }  
4  
5 function computeDiscount(price: number, rate: number): number {  
6     return price * (1 - rate); // function-like  
7 }
```

Why the difference matters in practice

When you mix side effects into computations, you make code harder to test and reason about.

Typical symptoms

- A function sometimes returns a value, sometimes prints or mutates global state
- Hidden dependencies (reads global variables, reads time, reads random)
- Unclear call order changes results

A common refactor is to separate computation from effects.

```
1 function computeTotal(items: number[]): number {  
2   return items.reduce((sum, x) => sum + x, 0);  
3 }  
4 function printReceipt(total: number): void {  
5   console.log("Total:", total);  
6 }  
7 const total = computeTotal([10, 20, 5]);  
8 printReceipt(total);
```

Java viewpoint: methods that return vs void methods

Java uses methods for both procedure-like and function-like behavior.

```
1 class Cart {  
2     private int totalCents = 0;  
3  
4     void addItem(int priceCents) {    // procedure-like  
5         totalCents += priceCents;  
6     }  
7  
8     int getTotalCents() {             // function-like  
9         return totalCents;  
10    }  
11 }
```

Design guideline

- Use void methods for state change and I/O
- Use returning methods for query-like operations

Python viewpoint: everything is an object, but intent still matters

Python does not require a separate procedure construct. Returning None is common for procedure-like functions.

```
1 def add_item(cart, price_cents):
2     cart["total"] += price_cents    # procedure-like mutation
3     # implicit return None
4
5 def total_cents(cart):
6     return cart["total"]            # function-like query
7
8 cart = {"total": 0}
9 add_item(cart, 199)
10 print(total_cents(cart))
```

First-Class Functions

- 1 Procedures and Functions
- 2 First-Class Functions**
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

First-class vs second-class functions

- First-class function means a function can be used like a value.
- Typical capabilities
 - ▶ Assign to variables
 - ▶ Pass as arguments
 - ▶ Return from other functions
 - ▶ Store in data structures
- Second-class function usually means some of those are restricted, or require special wrappers.
- This matters because many patterns in modern software rely on passing behavior around: callbacks, middleware, event handlers, and functional pipelines.

lambda

Lambda

- An anonymous function written inline without a name
- Used when a function is needed as a value (pass it, store it, return it)
- Often short; can capture variables from surrounding scope (closure)

Same idea in three languages:

```
1 | TypeScript: (x) => x * 2
2 | Python:      lambda x: x * 2
3 | Java:        x -> x * 2
```

Compare to a named function:

```
1 | function twice(x: number): number { return x * 2; }
2 | const twice2 = (x: number) => x * 2;
```

Both represent a function value; the lambda is just the unnamed form.

TypeScript: functions are first-class

In TypeScript, functions are values with types.

```
1  type Combiner = (a: number, b: number) => number;
2
3  const add: Combiner = (a, b) => a + b;
4  const mul: Combiner = (a, b) => a * b;
5
6  function apply(a: number, b: number, op: Combiner): number {
7      return op(a, b);
8  }
9
10 console.log(apply(2, 3, add));
11 console.log(apply(2, 3, mul));
```

Java: functions via interfaces and lambdas

Java historically emphasized methods, but modern Java supports lambdas targeting functional interfaces.

```
1 import java.util.function.IntBinaryOperator;
2 class Demo {
3     static int apply(int a, int b, IntBinaryOperator op) {
4         return op.applyAsInt(a, b);
5     }
6     public static void main(String[] args) {
7         IntBinaryOperator add = (x, y) -> x + y;
8         IntBinaryOperator mul = (x, y) -> x * y;
9
10        System.out.println(apply(2, 3, add));
11        System.out.println(apply(2, 3, mul));
12    }
13 }
```

The function value is represented through an object that implements an interface.

Cost model: why some languages needed wrappers

- In languages designed before first-class functions were common, subprograms were not naturally values.
- A common workaround pattern
 - ▶ Define an interface with one method
 - ▶ Pass objects implementing that interface
 - ▶ Later add lambda syntax as shorthand
- Consequence
 - ▶ Extra types and boilerplate compared to languages that treat functions as direct values

Parameters and Argument Passing

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing**
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

Parameter passing models

- Parameter passing describes how the arguments at the call site relate to the parameters in the callee.
- Common models (conceptual)
 - ▶ Pass-by-value: copy the value into the parameter
 - ▶ Pass-by-reference: parameter aliases the caller's variable
 - ▶ Pass-by-result (out): parameter writes back to caller on return
 - ▶ Pass-by-value-result (inout): copy in on call, copy back on return
- Real languages often implement combinations, and object references complicate intuition.

TypeScript: primitives vs objects at the call boundary

- TypeScript (JavaScript runtime) passes argument values.
- Key detail
 - ▶ For primitives, the value is the primitive itself
 - ▶ For objects, the value is a reference to the object

```
1 function bump(n: number): void {  
2     n = n + 1;  
3 }  
4 function rename(user: { name: string }): void {  
5     user.name = "new";  
6 }  
7 let x = 10;  
8 bump(x);  
9 console.log(x); // still 10  
10 let u = { name: "old" };  
11 rename(u);  
12 console.log(u.name); // "new"
```

How to describe this precisely

- A precise explanation that avoids confusion
 - ▶ The argument expression is evaluated to a value
 - ▶ That value is assigned to the parameter variable
 - ▶ Reassigning the parameter does not affect the caller variable
 - ▶ Mutating through an object reference affects the shared object
- Many programmers call this pass-by-sharing or call-by-object-sharing to highlight that references are shared, but variables are not aliased.

Practice 1: predict the output (TypeScript)

Read and predict the printed output, then explain using parameter passing and mutation rules.

```
1 function change(n: number, obj: { v: number }): void {  
2     n = n + 100;  
3     obj.v = obj.v + 100;  
4 }  
5  
6 let a = 1;  
7 let b = { v: 1 };  
8  
9 change(a, b);  
10  
11 console.log(a);  
12 console.log(b.v);
```

- Which assignment is local only, and which affects shared state
- How to refactor to avoid mutation

Practice 1 Answer: primitives vs objects (TypeScript)

```
1 function change(n: number, obj: { v: number }): void {
2     n = n + 100;
3     obj.v = obj.v + 100;
4 }
5 let a = 1; let b = { v: 1 };
6 change(a, b);
7 console.log(a); //1
8 console.log(b.v); // 101
```

- $n = n + 100$ is local only because n is a local parameter variable; it does not alias the caller variable a .
- $\text{obj.v} = \text{obj.v} + 100$ affects shared state because obj refers to the same object as b ; it mutates that object in place.

Refactor to avoid mutation (return new values instead of mutating)

```
1 function changePure(n: number, obj: { v: number }): { n2: number; obj2: { v:
  ↳ number } } {
2     return { n2: n + 100, obj2: { v: obj.v + 100 } };
3 }
4 let a = 1; let b = { v: 1 };
5 const { n2, obj2 } = changePure(a, b);
6 console.log(a); // 1
7 console.log(b.v); // 1
8 console.log(n2); // 101
9 console.log(obj2.v); // 101
```

Java passes argument values.

- Primitive arguments behave like pass-by-value
- Object arguments pass a reference value; mutation affects shared object
- Reassigning the parameter does not update the caller variable

```
1 class User {
2     String name;
3     User(String n) { name = n; }
4 }
5 class Demo {
6     static void reassign(User u) {
7         u = new User("new"); // only local parameter changes
8     }
9     static void mutate(User u) {
10        u.name = "mutated";
11    }
12    public static void main(String[] args) {
13        User a = new User("old");
14        reassign(a);
15        System.out.println(a.name); // "old"
16        mutate(a);
17        System.out.println(a.name); // "mutated"
18    }
19 }
```

Python: references and mutation

Python also binds names to objects. Calling a function binds parameter names to argument objects.

```
1 def reassign(lst):  
2     lst = [99]          # local name now points elsewhere  
3  
4 def mutate(lst):  
5     lst.append(99)      # mutates the shared object  
6  
7 a = [1, 2]  
8 reassign(a)  
9 print(a)               # [1, 2]  
10 mutate(a)  
11 print(a)               # [1, 2, 99]
```

Practical rule: reassignment is local; mutation is shared.

Pass-by-reference

- In pass-by-reference, the callee parameter is an alias for the caller variable.
- Consequences
 - ▶ Reassigning the parameter updates the caller variable
 - ▶ The callee can replace what the caller variable refers to
- Many mainstream languages avoid general pass-by-reference for safety and reasoning, but they may offer it in limited forms:
 - ▶ C++ references
 - ▶ C# ref and out
 - ▶ Rust borrowing rules

Simulating out parameters in TypeScript

TypeScript does not have out parameters, but you can use objects to carry results.

```
1  type ParseResult =  
2    { ok: boolean; value?: number; error?: string };  
3  
4  function parseIntSafe(s: string): ParseResult {  
5    const n = Number.parseInt(s, 10);  
6    if (Number.isNaN(n)) {  
7      return {ok: false, error: "not an int" };  
8    }  
9    return { ok: true, value: n };  
10 }  
11  
12 const r = parseIntSafe("123");  
13 if (r.ok) console.log(r.value);
```

Industry pattern: return a value object instead of mutating multiple outputs.

Multiple results: tuples in Python

Python commonly returns multiple values as a tuple.

```
1 def divmod_like(a, b):  
2     q = a // b  
3     r = a % b  
4     return q, r  
5  
6 q, r = divmod_like(17, 5)  
7 print(q, r)
```

Returning structured results often replaces out parameters in practice.

Multiple results: records in TypeScript

TypeScript commonly returns an object with named fields.

```
1 function divmodLike(a: number, b: number): { q: number; r:  
  ↪ number } {  
2   return { q: Math.floor(a / b), r: a % b };  
3 }  
4  
5 const { q, r } = divmodLike(17, 5);  
6 console.log(q, r);
```

Named fields make call sites self-documenting.

Defaults, Named Arguments, and Varargs

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs**
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

Default parameters in TypeScript

Default parameters apply when the argument is missing or undefined.

```
1 function greet(name: string, punctuation: string = "!"):
   ↪ string {
2     return "Hello, " + name + punctuation;
3 }
4
5 console.log(greet("Ada"));
6 console.log(greet("Ada", "!!"));
```

Common use: small configuration knobs without forcing the caller to pass every argument.

Named arguments pattern in TypeScript

TypeScript does not have true named arguments at call sites, but an options object provides similar readability.

```
1 type FetchOptions = { timeoutMs?: number; retries?:  
  ↪ number; cache?: boolean };  
2  
3 function fetchWithOptions(url: string, options:  
  ↪ FetchOptions = {}): void {  
4   const timeoutMs = options.timeoutMs ?? 2000;  
5   const retries = options.retries ?? 2;  
6   const cache = options.cache ?? true;  
7   console.log(url, timeoutMs, retries, cache);  
8 }  
9  
10 fetchWithOptions("https://example.com", { retries: 5,  
  ↪ cache: false });
```

Benefit: callers can specify only what they need, in any order.

Practice 2: implement a safe API with options objects

Goal: redesign a function signature to avoid positional confusion.

Starting point, hard to read at call sites:

```
1 function request(url: string, timeoutMs: number, retries:  
  ↪ number, cache: boolean): void {  
2   console.log(url, timeoutMs, retries, cache);  
3 }  
4  
5 request("/api/users", 2000, 3, false);
```

Problem with positional parameters

- `request("/api/users", 2000, 3, false)` is hard to read: the meaning of each number/boolean is not obvious.
- Callers can easily swap arguments accidentally.

Task: convert to an options object with defaults and show clearer call examples.

Practice 2 Answer

Refactor using an options object with defaults:

```
1 type RequestOptions = {  
2   timeoutMs?: number; retries?: number; cache?: boolean;  
3 };  
4 function request(url: string, opts: RequestOptions = {}): void  
5   ↪ {  
6     const timeoutMs = opts.timeoutMs ?? 2000;  
7     const retries = opts.retries ?? 2;  
8     const cache = opts.cache ?? true;  
9     console.log(url, timeoutMs, retries, cache);  
10  }  
11 request("/api/users");  
12 request("/api/users", { retries: 3, cache: false });  
13 request("/api/users", { timeoutMs: 5000 });
```

Why this is better

- The call site communicates intent through property names.
- Callers can specify only what they need.
- Adding a new option later does not force changes to every call site.

Python: default and named arguments

Python supports default values and keyword arguments directly.

```
1 def connect(host, port=5432, timeout_s=2.0):  
2     return (host, port, timeout_s)  
3  
4 print(connect("db.local"))  
5 print(connect("db.local", timeout_s=5.0))  
6 print(connect(host="db.local", port=6000))
```

Keyword arguments significantly improve readability for functions with many parameters.

Python pitfall: mutable default arguments

Default values are evaluated once when the function is defined, not each call.

```
1 def add_item(x, lst=[]):
2     lst.append(x)
3     return lst
4
5 print(add_item(1))    # [1]
6 print(add_item(2))    # [1, 2] surprising for many learners
```

Safe pattern uses None and creates a new list inside.

```
1 def add_item_safe(x, lst=None):
2     if lst is None:
3         lst = []
4     lst.append(x)
5     return lst
```

Java: default arguments via overloading

Java does not have default arguments; common substitutes are overloaded methods or builders.

```
1 class Connector {
2     static String connect(String host) {
3         return connect(host, 5432, 2.0);
4     }
5     static String connect(String host, int port) {
6         return connect(host, port, 2.0);
7     }
8
9     static String connect(String host, int port, double
10        ↪ timeoutS) {
11         return host + ":" + port + " timeout=" + timeoutS;
12     }
13 }
```

Overloads can grow quickly; builders are often preferred for many optional fields.

Variadic parameters: rest args

Variadic functions accept a variable number of arguments.

```
1 function sum(...xs: number[]): number {  
2   return xs.reduce((acc, x) => acc + x, 0);  
3 }  
4  
5 console.log(sum(1, 2, 3));  
6 console.log(sum(10));
```

Similar features exist in Java and Python, but the call syntax and typing differ.

Variadic parameters: Python and Java

Python uses star args.

```
1 def sum_all(*xs):
2     total = 0
3     for x in xs:
4         total += x
5     return total
6
7 print(sum_all(1, 2, 3))
```

Java uses varargs.

```
1 class Demo {
2     static int sumAll(int... xs) {
3         int total = 0;
4         for (int x : xs) total += x;
5         return total;
6     }
7 }
```

Closures and Capturing

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing**
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

Closures and lexical environments

Closure

- A function together with the environment of variables it captures
- The captured variables are free variables in the function body (not parameters, not locals)

Lexical environment

- The set of bindings visible due to where the function is defined in source code
- Lexical scoping means the structure of the code determines variable resolution

Closures allow behavior to depend on values that existed when the function was created, even if the outer function has returned.

TypeScript closure: function returning function

This example returns a function that remembers a captured variable.

```
1 function makeAdder(k: number): (x: number) => number {  
2   return (x: number) => x + k; // captures k  
3 }  
4  
5 const add10 = makeAdder(10);  
6 const add3 = makeAdder(3);  
7  
8 console.log(add10(5)); // 15  
9 console.log(add3(5)); // 8
```

k remains available because the returned function closes over it.

Practice 3: function factory and capture (TypeScript)

```
1 function makeLimiter(max: number): (x: number) => number {  
2   // return a function that clamps x into [0, max]  
3   return (x: number) => {  
4     if (x < 0) return 0;  
5     if (x > max) return max;  
6     return x;  
7   };  
8 }  
9 const limit10 = makeLimiter(10);  
10 console.log(limit10(-3));  
11 console.log(limit10(7));  
12 console.log(limit10(100));
```

Discussion: what variable is captured, and how does it affect behavior?

Practice 3: function factory and capture (TypeScript)

```
1 function makeLimiter(max: number): (x: number) => number {  
2     return (x: number) => {  
3         if (x < 0) return 0;  
4         if (x > max) return max;  
5         return x;  
6     };  
7 }  
8 const limit10 = makeLimiter(10);  
9 console.log(limit10(-3)); // 0  
10 console.log(limit10(7)); // 7  
11 console.log(limit10(100)); // 10
```

What variable is captured, and how does it affect behavior

- The returned function closes over max.
- max is fixed when makeLimiter is called; for limit10 it is 10.
- Each call to limit10 uses the same captured max value to clamp the input.
- A different call makeLimiter(5) would create a different returned function with a different captured max.

Python closure: nested function capturing outer variable

Python closures behave similarly.

```
1 def make_adder(k):  
2     def add(x):  
3         return x + k    # captures k  
4     return add  
5  
6 add10 = make_adder(10)  
7 print(add10(5))
```

Nested definitions are a common way to build specialized behavior.

Java closure-like behavior with lambdas

Java lambdas can capture variables, with a restriction.

Captured local variables must be effectively final (not reassigned).

```
1 import java.util.function.IntUnaryOperator;
2
3 class Demo {
4     static IntUnaryOperator makeAdder(int k) {
5         return (int x) -> x + k; // captures k
6     }
7
8     public static void main(String[] args) {
9         IntUnaryOperator add10 = makeAdder(10);
10        System.out.println(add10.applyAsInt(5));
11    }
12 }
```

The restriction simplifies reasoning about captured locals and threading.

What gets captured: value vs variable

- A key subtlety: closures capture variables (bindings), not just values.
- Consequences
 - ▶ If the variable changes later, the closure observes the updated value
 - ▶ Captured variables can extend lifetimes of data beyond the original scope
- This is powerful but can cause surprising bugs, especially with loops and asynchronous callbacks.

TypeScript: capturing and mutation

The closure observes changes to captured variables.

```
1 function makeCounter(): () => number {  
2     let count = 0;  
3     return () => {  
4         count = count + 1;  
5         return count;  
6     };  
7 }  
8  
9 const c = makeCounter();  
10 console.log(c()); // 1  
11 console.log(c()); // 2
```

This pattern implements private state without a class.

Python: closure with nonlocal for reassignment

Python requires nonlocal to rebind a captured variable.

```
1 def make_counter():
2     count = 0
3     def inc():
4         nonlocal count
5         count += 1
6         return count
7     return inc
8
9 c = make_counter()
10 print(c())
11 print(c())
```

Without nonlocal, count would be treated as a new local inside inc.

Closure lifetime and memory

When an inner function outlives the outer function call, captured variables must remain alive.

Practical implications

- Closures can keep large objects alive longer than intended
- Event handlers can accidentally retain DOM nodes and leak memory
- Long-lived closures should capture only what they need

A useful habit: avoid capturing entire objects when a small primitive value is enough.

Closure Pitfalls

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls**
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

Classic bug: loop variable capture in JavaScript-style code

The bug often appears when callbacks run later, after the loop finishes.

```
1  const callbacks: Array<() => void> = [];  
2  
3  for (var i = 0; i < 3; i++) {  
4      callbacks.push(() => console.log(i));  
5  }  
6  
7  callbacks[0]();  
8  callbacks[1]();  
9  callbacks[2]();
```

With *var*, *i* is function-scoped, so all callbacks share the same *i*.

Fix: use block scope with let

let creates a new binding per iteration in a for loop, so each closure captures a distinct variable.

```
1  const callbacks: Array<() => void> = [];  
2  
3  for (let i = 0; i < 3; i++) {  
4      callbacks.push(() => console.log(i));  
5  }  
6  
7  callbacks[0](); // 0  
8  callbacks[1](); // 1  
9  callbacks[2](); // 2
```

Rule: prefer let and const over var in modern TypeScript.

Python: late binding in closures inside loops

Python closures in loops can surprise you similarly.

```
1 | funcs = []  
2 | for i in range(3):  
3 |     funcs.append(lambda: i)  
4 |  
5 | print(funcs[0]()) #2  
6 | print(funcs[1]()) #2  
7 | print(funcs[2]()) #2
```

Many learners expect 0, 1, 2, but all lambdas refer to the same `i` binding.

Python fix: capture current value using a default argument

Default arguments are evaluated at function creation time, capturing the current loop value.

```
1 | funcs = []  
2 | for i in range(3):  
3 |     funcs.append(lambda i=i: i)  
4 |  
5 | print(funcs[0]()) # 0  
6 | print(funcs[1]()) # 1  
7 | print(funcs[2]()) # 2
```

This is a widely used idiom for loop capture in Python.

Practice 4: closure and loop capture (Python)

Predict the output and propose a fix.

```
1 funcs = []  
2 for i in range(4):  
3     def f():  
4         return i  
5     funcs.append(f)  
6  
7 for g in funcs:  
8     print(g())
```

- What is the captured variable
- When is the variable value read
- Fix using default-argument capture or another approach

Practice 4 Answer: closure and loop capture (Python)

Which value is captured, and when is it read

- The captured variable is `i` from the surrounding scope (the loop variable).
- The value of `i` is read when `f` is called later (`g()`), not when `f` is defined.
- After the loop ends, `i` equals 3, so all stored functions return 3.

```
1 funcs = [] #Fix using default-argument capture:
2 for i in range(4):
3     def f(i=i):
4         return i
5     funcs.append(f)
6 for g in funcs:
7     print(g())

1 #fix using a helper function create a new scope
2 def make_f(i):
3     def f():
4         return i
5     return f
6 funcs = []
7 for i in range(4):
8     funcs.append(make_f(i))
```

Higher-Order Functions and Callbacks

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks**
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns

Higher-order functions

A higher-order function is a function that

- takes a function as an argument, or
- returns a function as a result

Benefits

- Code reuse through generic control patterns (map, filter, retry, debounce)
- Separation of policy from mechanism (algorithm vs decision rule)
- Composition: build behavior by combining functions

Costs

- Debugging can be harder due to indirect control flow
- Types and error handling need careful design

Callback

- A function you pass into another function so the other function can call it
- Inversion of control: the library controls when/how many times it calls your function
- Can be called immediately (synchronous) or later (asynchronous)

Synchronous callback example (array processing):

```
1 | const nums = [1, 2, 3];  
2 | const squares = nums.map(x => x * x); // callback is x => x * x
```

Asynchronous callback example (event-style):

```
1 | function onClick(handler: () => void): void {  
2 |   // real UI code would call handler later  
3 |   handler();  
4 | }  
5 | onClick(() => console.log("clicked")); // callback is () => ...
```

A callback API should document when it runs, how often, and how errors are reported.

TypeScript: map, filter, reduce as higher-order functions

Arrays provide higher-order functions that accept callbacks.

```
1  const nums = [1, 2, 3, 4, 5];  
2  
3  const squares = nums.map(x => x * x);  
4  const evens = nums.filter(x => x % 2 === 0);  
5  const sum = nums.reduce((acc, x) => acc + x, 0);  
6  
7  console.log(squares);  
8  console.log(evens);  
9  console.log(sum);
```

The callback provides the custom behavior; the loop mechanism is reused.

Python: list processing with functions

Python offers functional tools and comprehensions.

```
1 | nums = [1, 2, 3, 4, 5]
2 |
3 | squares = list(map(lambda x: x * x, nums))
4 | evens = list(filter(lambda x: x % 2 == 0, nums))
5 | total = sum(nums)
6 |
7 | print(squares)
8 | print(evens)
9 | print(total)
```

In Python, comprehensions are often preferred for readability, but callbacks remain important for libraries.

Java: higher-order style with streams

Java streams accept lambdas and functional interfaces.

```
1 import java.util.List;
2
3 class Demo {
4     public static void main(String[] args) {
5         List<Integer> nums = List.of(1, 2, 3, 4, 5);
6
7         int sum = nums.stream()
8             .filter(x -> x % 2 == 0)
9             .map(x -> x * x)
10            .reduce(0, (acc, x) -> acc + x);
11
12        System.out.println(sum);
13    }
14 }
```

Streams encourage a pipeline model where each step is a small function.

Callbacks: event handler example

UI and server frameworks often use callbacks because they invert control.

```
1 function onClick(handler: () => void): void {  
2     // imagine this registers handler with a UI system  
3     handler(); // demo call  
4 }  
5  
6 let clicks = 0;  
7  
8 onClick(() => {  
9     clicks = clicks + 1;  
10    console.log("clicks", clicks);  
11 });
```

The callback closes over clicks, so it can update private state across events.

Callback design considerations

When you expose callbacks in an API, callers need to know:

- When the callback is invoked (immediately, later, possibly multiple times)
- Which thread or event loop invokes it
- Error signaling model (throw, return value, error parameter, promise rejection)
- Cancellation mechanism

Many production issues happen because the callback contract is unclear, not because the syntax is hard.

Async Patterns in Industry

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry**
- 9 Closures in UI and React
- 10 Closure Design Patterns

Industry angle: why callback hell happens

Callback hell is deeply nested callbacks that appear when:

- Each async step depends on the previous step result
- Errors are handled separately at each nesting level
- Multiple branches and retries are needed

Symptoms

- Hard to read control flow
- Duplication of error handling
- Difficult to compose and test

Modern JavaScript ecosystems introduced Promises and `async/await` to make async flow look like sequential code.

TypeScript: callback-based async style

This style is still common in older Node.js APIs and some libraries.

```
1 type Callback<T> = (err: Error | null, value?: T) => void;
2 function readUser(id: string, cb: Callback<{ id: string; name:
  ↳ string }>): void {
3     setTimeout(() => cb(null, { id, name: "Ada" }), 10);
4 }
5 function readOrders(userId: string, cb: Callback<string[]>):
  ↳ void {
6     setTimeout(() => cb(null, ["A1", "B2"]), 10);
7 }
8 readUser("u1", (err, user) => {
9     if (err || !user) return console.log("error", err);
10    readOrders(user.id, (err2, orders) => {
11        if (err2 || !orders) return console.log("error", err2);
12        console.log(user.name, orders);
13    });
14 });
```

Nesting grows as you add more steps, branching, and retries.

TypeScript: Promises reduce nesting

Promises represent a value that may be available later.

```
1 function readUserP(id: string): Promise<{ id: string; name:  
  ↪ string }> {  
2   return new Promise(resolve => setTimeout(() => resolve({ id,  
  ↪ name: "Ada" }), 10));  
3 }  
4  
5 function readOrdersP(userId: string): Promise<string[]> {  
6   return new Promise(resolve => setTimeout(() => resolve(["A1",  
  ↪ "B2"]), 10));  
7 }  
8  
9 readUserP("u1")  
10  .then(user => readOrdersP(user.id).then(orders => [user,  
  ↪ orders] as const))  
11  .then(([user, orders]) => console.log(user.name, orders))  
12  .catch(err => console.log("error", err));
```

Control flow is flatter, but chains can still become complex.

TypeScript: async/await for sequential readability

async/await is syntax over Promises that improves readability.

```
1  async function workflow(): Promise<void> {
2      try {
3          const user = await readUserP("u1");
4          const orders = await readOrdersP(user.id);
5          console.log(user.name, orders);
6      } catch (err) {
7          console.log("error", err);
8      }
9  }
10 workflow();
```

try/catch works for awaited promise rejections, which simplifies error handling.

Practice 5: refactor callback nesting to async/await

Starting point:

```
1 type Callback<T> = (err: Error | null, value?: T) => void;
2 function step1(cb: Callback<number>): void {
3   setTimeout(() => cb(null, 10), 10);
4 }
5 function step2(x: number, cb: Callback<number>): void {
6   setTimeout(() => cb(null, x + 5), 10);
7 }
8 function step3(x: number, cb: Callback<number>): void {
9   setTimeout(() => cb(null, x * 2), 10);
10 }
11 step1((e1, a) => {
12   if (e1 || a === undefined) return console.log("error", e1);
13   step2(a, (e2, b) => {
14     if (e2 || b === undefined) return console.log("error", e2);
15     step3(b, (e3, c) => {
16       if (e3 || c === undefined) return console.log("error", e3);
17       console.log("result", c);
18     });
19   });
20 });
```

Task: rewrite using Promises and async/await while preserving error handling.

Practice 5 Answer: refactor callback nesting to async/await (TypeScript)

What the original code computes if no errors occur

- step1 produces 10
- step2 adds 5, producing 15
- step3 multiplies by 2, producing 30
- printed line is result 30

Promise wrappers (convert error-first callbacks into Promises):

```
1 type Callback<T> = (err: Error | null, value?: T) => void;
2
3 function step1(cb: Callback<number>): void {
4     setTimeout(() => cb(null, 10), 10);
5 }
6 function step2(x: number, cb: Callback<number>): void {
7     setTimeout(() => cb(null, x + 5), 10);
8 }
9 function step3(x: number, cb: Callback<number>): void {
10    setTimeout(() => cb(null, x * 2), 10);
```

Async closures: what is captured

In async code, closures often capture variables whose values change over time.

Typical bugs

- Capturing a loop variable and using it when the async task resolves
- Capturing a mutable object and seeing later changes unexpectedly

Typical fixes

- Use block scope variables for each iteration
- Capture immutable snapshots (copy the needed values)
- Prefer pure functions and explicit parameters for async callbacks

Closures in UI and React

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React**
- 10 Closure Design Patterns

Closures in UI: event handlers

UI systems store callback functions and invoke them later.

That means:

- The handler must keep access to needed state
- Closures provide that access
- Captured state may become stale if the UI updates but the handler closes over an old snapshot

This is a common source of bugs in React and other event-driven frameworks.

React hook idea: stale closure pattern (conceptual TypeScript)

This is a simplified illustration of a stale capture issue.

```
1 | let count = 0;
2 |
3 | function makeHandler() {
4 |   const captured = count;
5 |   return () => console.log("captured", captured,
   |     ↪ "current", count);
6 | }
7 |
8 | const h1 = makeHandler();
9 | count = 5;
10| h1(); // captured 0, current 5
```

In React, handlers created during a render capture that render's state values.

React-style fix: functional update pattern (conceptual TypeScript)

A common fix is to compute next state from previous state, avoiding dependence on a captured stale value.

```
1 function updateCount(prev: number): number {  
2   return prev + 1;  
3 }  
4  
5 let state = 0;  
6 state = updateCount(state);  
7 state = updateCount(state);  
8 console.log(state);
```

React provides a `setState` form that takes a function `prev → next` to address stale closure issues.

Rule of thumb for closures in production code

Prefer explicit parameters for data that varies per call.

Use closures for

- Configuration fixed at creation time
- Encapsulated private state (counters, memoization caches)
- Dependency injection (passing services without globals)

Avoid closures that capture

- Large objects unnecessarily
- Variables that change frequently when you need the latest value
- Loop indices used later in async callbacks without proper scoping

Argument evaluation order and side effects

When a call has multiple argument expressions, the language defines an order.

Why this matters

- If argument expressions have side effects, order changes behavior
- Code becomes fragile and hard to refactor

Practical guidance

- Avoid side effects inside arguments
- Compute intermediate values in separate statements for clarity

TypeScript: make side effects explicit

Instead of embedding state changes into call arguments, separate them.

```
1  let i = 0;
2
3  function f(a: number, b: number): void {
4      console.log("a", a, "b", b);
5  }
6
7  i = i + 1;
8  const a = i;
9  i = i + 1;
10 const b = i;
11
12 f(a, b);
```

This style is easier to read and debug than relying on tricky evaluation order.

Closure Design Patterns

- 1 Procedures and Functions
- 2 First-Class Functions
- 3 Parameters and Argument Passing
- 4 Defaults, Named Arguments, and Varargs
- 5 Closures and Capturing
- 6 Closure Pitfalls
- 7 Higher-Order Functions and Callbacks
- 8 Async Patterns in Industry
- 9 Closures in UI and React
- 10 Closure Design Patterns**

Closure pattern: memoization

Memoization caches results based on inputs to speed up repeated calls.

```
1 function memoize1<A, R>(f: (a: A) => R): (a: A) => R {
2   const cache = new Map<A, R>();
3   return (a: A) => {
4     if (cache.has(a)) return cache.get(a)!;
5     const r = f(a);
6     cache.set(a, r);
7     return r;
8   };
9 }
10
11 const slowSquare = (x: number) => x * x;
12 const fastSquare = memoize1(slowSquare);
13 console.log(fastSquare(10));
14 console.log(fastSquare(10));
```

The cache is private state captured by the returned function.

Closure pattern: debounce for event handlers

Debounce delays execution until events stop firing for a period.

```
1 function debounce(fn: () => void, delayMs: number): () => void
  ↪ {
2   let timer: ReturnType<typeof setTimeout> | undefined;
3   return () => {
4     if (timer !== undefined) clearTimeout(timer);
5     timer = setTimeout(() => fn(), delayMs);
6   };
7 }
8 let fired = 0;
9 const handler = debounce(() => {
10   fired = fired + 1;
11   console.log("fired", fired);
12 }, 200);
13 handler();
14 handler();
15 handler(); // only one execution after the last call settles
```

timer is captured state that persists across invocations.

Closure pattern: dependency injection

Instead of reading globals, create a function that closes over dependencies.

Example concept

- Provide a logger, configuration, or database client once
- Return a handler that uses those dependencies without passing them every time

Benefits

- Easier testing with fake dependencies
- Fewer hidden global dependencies
- Better modularity

Dependency injection with closures in TypeScript

Create a specialized function by capturing a service.

```
1 type Logger = { log: (msg: string) => void };
2
3 function makeGreeter(logger: Logger): (name: string) => void {
4     return (name: string) => logger.log("hello " + name);
5 }
6
7 const consoleLogger: Logger = {log: (msg) => console.log(msg)};
8 const greet = makeGreeter(consoleLogger);
9
10 greet("Ada");
```

The returned function depends on logger without using a global variable.

Java: capturing effectively final variables

In Java, a lambda is allowed to capture a local variable only if that variable is final or effectively final.

```
1 import java.util.function.IntUnaryOperator;
2 class Demo {
3     static IntUnaryOperator makeScaler(int factor) {
4         return x -> x * factor; //factor is effectively final
5     }
6     public static void main(String[] args) {
7         IntUnaryOperator times3 = makeScaler(3);
8         System.out.println(times3.applyAsInt(7));
9     }
10 }
```

If factor were reassigned later in the same scope, the code would not compile.

```
1 static IntUnaryOperator makeScaler(int factor) {
2     factor = factor + 1;           // reassignment
3     return x -> x * factor;       // compile error
4 }
```

Python: capturing configuration for callbacks

Capturing configuration is a common pattern for callbacks.

```
1 def make_prefix_logger(prefix):
2     def log(msg):
3         print(prefix + ": " + msg)
4     return log
5
6 info = make_prefix_logger("INFO")
7 warn = make_prefix_logger("WARN")
8
9 info("server started")
10 warn("disk almost full")
```

Each returned function retains its own prefix.

Summary: what to remember

Key takeaways

- Procedures emphasize actions and effects; functions emphasize computed results
- TypeScript and Python have first-class functions directly; Java uses functional interfaces and lambdas
- Many languages pass argument values; references can make mutation visible across call boundaries
- Default parameters and named arguments improve call-site readability (Python directly, TypeScript via defaults and options objects, Java via overloads/builders)
- A closure is a function plus captured lexical environment; capture can extend lifetimes and cause subtle bugs
- Callback-heavy async code can become hard to manage; Promises and `async/await` improve structure

Review questions

Use these to check understanding

- In TypeScript, why does mutating an object parameter affect the caller, but reassigning the parameter does not?
- What does effectively final mean in Java, and why does it matter for lambdas?
- In Python, why does a lambda inside a loop often returns the last loop value?
- When should you prefer an options object over positional parameters?
- How can closures lead to memory leaks in event-driven code?
- What changes when you refactor a nested callback flow into `async/await`?